

SvelteKit + Bun: From Dev Container to Production on Your VPS

A SvelteKit app running on Bun, developed in a VS Code Dev Container, built into a Docker image, pushed to GHCR, deployed to your VPS — the same pipeline as [dockerized-deployments](#), but specifically wired for the SvelteKit + Bun stack. The TypeScript-side companion to that doc.

NOTE. LAST VALIDATED: 2026-05. SvelteKit 2.x with Svelte 5 (runes), Bun 1.1+, [@sveltejs/adapter-node](#) current, [postgres](#) (the postgres.js client) 3.x, Drizzle ORM 0.30+, Lucia v3 / Better Auth. Bump and re-validate annually per [CLAUDE.md](#) standard #5.

What you'll have at the end

- A SvelteKit project scaffolded with Bun.
- A multi-stage Dockerfile with a **dev** stage (for the Dev Container) and a **runtime** stage (the minimal image that ships).
- A VS Code Dev Container that runs Bun + your code + a local Postgres for development.
- The same `build.yml` / `deploy.yml` pipeline as the Python guide, adapted for SvelteKit + Bun.
- Caddy serving HTTPS for your SvelteKit app over the shared network.
- Database integration via `postgres.js` + Drizzle for type-safe queries.
- Server-side env vars handled correctly (build-time vs runtime, public vs private).
- Auth wired with Lucia or Better Auth — the modern SvelteKit auth options.

Prerequisites

- VPS provisioned per [vps-from-zero](#) (Docker, deploy user, shared network, shared Postgres).
 - The [dockerized-deployments](#) pipeline already understood from that guide.
 - A GitHub repo for the new project.
 - Bun installed locally (for scaffolding): `curl -fsSL https://bun.sh/install | bash`.
-

Table of contents

1. [The shape: one image, dev container, prod, all on Bun](#)
2. [Scaffold the project](#)
3. [The multi-stage Dockerfile \(Bun edition\)](#)
4. [.dockerignore, .env.example, repo layout](#)
5. [Local docker-compose with Postgres](#)
6. [VS Code Dev Container](#)
7. [Database access: postgres.js + Drizzle](#)
8. [Auth: Lucia or Better Auth](#)

9. [Env vars: build-time vs runtime, public vs private](#)
 10. [CI: GitHub Actions → GHCR](#)
 11. [CD: deploying to the VPS](#)
 12. [Caddy site block for SvelteKit](#)
 13. [Troubleshooting](#)
 14. [Alternatives considered](#)
 15. [Quick reference](#)
-

1. The shape: one image, dev container, prod, all on Bun

The principles from [dockerized-deployments](#) all carry over. What changes is the language stack inside the container: instead of Python + uv, it's TypeScript + Bun.

[Diagram: see the online version on GitHub for the interactive flowchart.]

Two specifics worth noting:

- **Bun is the JavaScript runtime in both dev and prod.** No Node anywhere. SvelteKit's `adapter-node` produces a standard Node-style entrypoint that Bun runs without modification (Bun is Node-API-compatible).
 - **Same shared network and Postgres** as the Python apps. The SvelteKit container reaches Postgres at `postgres:5432`, just like any other app in the stack.
-

2. Scaffold the project

Create a new project locally (or directly in WSL):

```
cd ~/dev
bun create svelte@latest my-app
cd my-app
```

The interactive prompt:

Question	Pick
Which Svelte app template?	Skeleton project (we want minimal, not a demo)
Add type checking with TypeScript?	Yes, using TypeScript syntax
Add ESLint?	Yes
Add Prettier?	Yes
Add Playwright for E2E tests?	Optional — yes if you'll write tests, no if not
Add Vitest for unit tests?	Yes

Install dependencies:

```
bun install
```

Confirm it runs:

```
bun run dev
# > vite dev
# Local: http://localhost:5173/
```

Initialize a git repo and push:

```
git init
git add -A
git commit -m "Initial SvelteKit + Bun scaffold"
gh repo create my-app --private --source=. --remote=origin --push
```

Add the Node adapter (the project starts with `adapter-auto`, which guesses based on environment — for self-hosting we want explicit):

```
bun add -d @sveltejs/adapter-node
```

Update `svelte.config.js`:

```
import adapter from '@sveltejs/adapter-node';
import { vitePreprocess } from '@sveltejs/vite-plugin-svelte';

const config = {
  preprocess: vitePreprocess(),
  kit: {
    adapter: adapter({
      out: 'build',
      precompress: true,           // gzip + brotli the static assets at build time
      envPrefix: 'PUBLIC_'
    })
  }
};

export default config;
```

Remove `@sveltejs/adapter-auto`:

```
bun remove @sveltejs/adapter-auto
```

Confirm the build still works:

```
bun run build
ls build/           # should see index.js, env.js, server/, client/
```

`build/index.js` is the entrypoint — `bun run build/index.js` starts the production server on port 3000 by default.

3. The multi-stage Dockerfile (Bun edition)

```
# syntax=docker/dockerfile:1.7

# — Base stage: shared dependencies —————
FROM oven/bun:1.1-alpine AS base

WORKDIR /app

# Copy lockfile + package.json first for cache friendliness
COPY package.json bun.lockb ./
RUN bun install --frozen-lockfile --production=false

# — Dev stage: adds dev tools and the dev server entry —————
FROM base AS dev

# Add minimal apt tools (Alpine equivalents) for in-container dev work
RUN apk add --no-cache git curl openssh-client postgresql-client

# Source isn't copied – dev container mounts it as a volume
CMD ["bun", "run", "dev", "--", "--host", "0.0.0.0"]

# — Build stage: produce the production artifact —————
FROM base AS build

COPY . .
RUN bun run build

# Strip out dev dependencies
RUN bun install --frozen-lockfile --production

# — Runtime stage: minimal image for production —————
FROM oven/bun:1.1-alpine AS runtime

WORKDIR /app

# Just the built output and runtime node_modules (production)
COPY --from=build /app/build ./build
COPY --from=build /app/node_modules ./node_modules
COPY --from=build /app/package.json ./package.json

# Non-root user
RUN adduser -D -h /app app
USER app

ENV NODE_ENV=production
ENV PORT=3000
EXPOSE 3000

CMD ["bun", "run", "build/index.js"]
```

Why each stage

base — installs all dependencies (production + dev) for the dev container. We need devDeps in dev because `vite`, `prettier`, `eslint`, etc. all live there.

dev — adds OS tools for in-container debugging. Sets the dev server as the default command. The Dev Container will override this to keep the container alive idle.

build — runs the SvelteKit build. Output goes to `build/`. Then re-installs deps with `--production` to drop devDeps from `node_modules/`, shrinking the final image.

runtime — copies only the build output + production `node_modules` + `package.json`. Runs as non-root `app` user. Final image is ~80-120MB depending on dependencies — much smaller than a Node-based equivalent would be.

IMPORTANT. USE `OVEN/BUN:1.1-ALPINE`, NOT `OVEN/BUN:LATEST`. Pin the major version (`1.1`). Bun is moving fast; `latest` could ship a breaking change that surfaces only in CI.

Layer caching strategy

Same pattern as the Python doc: copy `package.json` + `bun.lockb` before the source, so dependency changes are the only thing that invalidates the install layer. Code changes don't trigger reinstalls.

4. `.dockerignore`, `.env.example`, repo layout

`.dockerignore`:

```
# secrets
.env
.env.*
!.env.example

# version control
.git
.gitignore

# node / build
node_modules
.svelte-kit
build
.vercel
.netlify

# editor
.vscode
.idea
*.swp
*.log

# test artifacts
coverage
playwright-report
test-results

# docs / meta
*.md
!README.md
```

`.env.example`:

```
# Database
DATABASE_URL=postgres://appuser:dev@postgres:5432/appdb

# Auth
AUTH_SECRET=replace-with-output-of-`openssl rand -hex 32`
PUBLIC_BASE_URL=http://localhost:5173

# Add others as needed
```

Repo structure after scaffolding:

```
my-app/
├── .devcontainer/
│   └── devcontainer.json
├── .github/
│   └── workflows/
│       ├── build.yml
│       └── deploy.yml
├── src/
│   ├── routes/
│   │   ├── +layout.svelte
│   │   ├── +page.svelte
│   │   └── api/
│   ├── lib/
│   │   ├── server/           # server-only code (DB, auth, secrets)
│   │   └── components/      # shared client components
│   ├── app.d.ts
│   └── app.html
├── static/
├── tests/
├── .dockerignore
├── .env.example
├── .env                      # gitignored
├── .gitignore
├── bun.lockb
├── docker-compose.yml       # local dev only
├── Dockerfile
├── drizzle.config.ts
├── package.json
├── playwright.config.ts
├── README.md
├── svelte.config.js
├── tsconfig.json
└── vite.config.ts
```

5. Local docker-compose with Postgres

`docker-compose.yml`:

```

services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: dev
      POSTGRES_DB: appdb
    ports:
      - "5432:5432"
    volumes:
      - postgres_dev_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U appuser -d appdb"]
      interval: 5s
      timeout: 3s
      retries: 5

  app:
    build:
      context: .
      target: dev
    env_file: .env
    ports:
      - "5173:5173"          # Vite dev server
    volumes:
      - ./app                # host source → container
      - /app/node_modules    # keep container's node_modules
      - /app/.svelte-kit     # keep container's .svelte-kit
    depends_on:
      postgres:
        condition: service_healthy
    command: bun run dev --host 0.0.0.0

  migrate:
    build:
      context: .
      target: dev
    profiles: ["migrate"]
    env_file: .env
    depends_on:
      postgres:
        condition: service_healthy
    command: bun run db:migrate

volumes:
  postgres_dev_data:

```

Why the named-volume tricks

`- /app/node_modules` and `- /app/.svelte-kit` (without a source path) prevent the host's mounted source from clobbering the container's `node_modules` and `.svelte-kit`. The container's installs (Linux/Alpine binaries) shouldn't get overwritten by your host's installs (macOS/Windows binaries that don't run inside Alpine).

This is a Bun + multi-OS development idiom. Without these "anonymous volume" overlays, every container start would clobber the installed deps with whatever your host has.

Why expose 5173

Vite's dev server. With `--host 0.0.0.0`, it binds to all interfaces inside the container, so the host can reach it on `localhost:5173`.

6. VS Code Dev Container

`.devcontainer/devcontainer.json`:

```
{
  "name": "my-app dev",
  "dockerComposeFile": ["../docker-compose.yml"],
  "service": "app",
  "workspaceFolder": "/app",
  "shutdownAction": "stopCompose",

  "build": {
    "target": "dev"
  },

  "overrideCommand": true,
  "customizations": {
    "vscode": {
      "extensions": [
        "svelte.svelte-vscode",
        "bradlc.vscode-tailwindcss",
        "esbenp.prettier-vscode",
        "dbaeumer.vscode-eslint",
        "ms-azuretools.vscode-docker",
        "mtxr.sqltools",
        "mtxr.sqltools-driver-pg"
      ],
      "settings": {
        "editor.formatOnSave": true,
        "editor.defaultFormatter": "esbenp.prettier-vscode",
        "[svelte]": {
          "editor.defaultFormatter": "svelte.svelte-vscode"
        },
        "typescript.tsdk": "node_modules/typescript/lib"
      }
    }
  },

  "forwardPorts": [5173, 5432],
  "remoteUser": "root"
}
```

Same template as the Python doc, but with Svelte / Bun-specific extensions and TS config.

Reopen in container

`Ctrl-Shift-P` → "Dev Containers: Reopen in Container." First build is 1-3 minutes; subsequent are seconds. Once attached, `bun run dev` runs SvelteKit's dev server on `http://localhost:5173/`, hot-reloading on file changes.

Hot reload

Vite's HMR (Hot Module Replacement) works inside the container as long as you have the volume mount (`./app`) in compose. Saving a `.svelte` file updates the browser without a full reload.

TIP. IF HMR SEEMS FLAKY ON WSL, it's the filesystem-watching issue: inotify events from `/mnt/c/...` are unreliable. Keep the repo in WSL's native filesystem (`/home/julian/dev/`) — not `/mnt/c/...` — and HMR works fine. (This is the same WSL guidance as elsewhere in CLAUDE.md.)

7. Database access: postgres.js + Drizzle

The Bun-friendly stack for Postgres:

- **postgres** (also known as postgres.js) — the fastest Node-API Postgres client. Bun-compatible.
- **drizzle-orm** — TypeScript-first ORM. Type-safe queries, lightweight, no runtime overhead. Schema-first migrations.

```
bun add postgres drizzle-orm
bun add -d drizzle-kit
```

Define the schema

`src/lib/server/db/schema.ts`:

```
import { pgTable, serial, text, timestamp, integer } from 'drizzle-orm/pg-core';

export const users = pgTable('users', {
  id: serial('id').primaryKey(),
  email: text('email').notNull().unique(),
  name: text('name').notNull(),
  createdAt: timestamp('created_at').defaultNow().notNull()
});

export const scores = pgTable('scores', {
  id: serial('id').primaryKey(),
  userId: integer('user_id').notNull().references(() => users.id),
  value: integer('value').notNull(),
  createdAt: timestamp('created_at').defaultNow().notNull()
});

export type User = typeof users.$inferSelect;
export type NewUser = typeof users.$inferInsert;
```

The client

`src/lib/server/db/index.ts`:

```
import { drizzle } from 'drizzle-orm/postgres-js';
import postgres from 'postgres';
import { DATABASE_URL } from '$env/static/private';
import * as schema from './schema';

const client = postgres(DATABASE_URL);
export const db = drizzle(client, { schema });
```

Drizzle Kit config

`drizzle.config.ts`:

```
import { defineConfig } from 'drizzle-kit';

export default defineConfig({
  schema: './src/lib/server/db/schema.ts',
  out: './drizzle',
  dialect: 'postgresql',
  dbCredentials: {
    url: process.env.DATABASE_URL!
  }
});
```

Scripts in package.json

```
{
  "scripts": {
    "dev": "vite dev",
    "build": "vite build",
    "preview": "vite preview",
    "check": "svelte-kit sync && svelte-check --tsconfig ./tsconfig.json",
    "lint": "prettier --check . && eslint .",
    "format": "prettier --write .",
    "test": "vitest",
    "test:e2e": "playwright test",
    "db:generate": "drizzle-kit generate",
    "db:migrate": "drizzle-kit migrate",
    "db:studio": "drizzle-kit studio"
  }
}
```

Generating and applying migrations

```
# After editing schema.ts, generate a migration SQL file
bun run db:generate
# This puts SQL in ./drizzle/ - review it before applying

# Apply migrations (against the configured DATABASE_URL)
bun run db:migrate
```

For the deploy pipeline (same pattern as the Python doc's Alembic migrations): a `migrate` service in compose that runs `bun run db:migrate` once and exits. The deploy workflow runs `docker compose run --rm migrate` before `up -d` for the app.

Using the DB in a route

`src/routes/+page.server.ts`:

```
import { db } from '$lib/server/db';
import { users } from '$lib/server/db/schema';
import { desc } from 'drizzle-orm';

export const load = async () => {
  const recentUsers = await db
    .select()
    .from(users)
    .orderBy(desc(users.createdAt))
    .limit(10);

  return { users: recentUsers };
};
```

Fully type-safe — the return type of `db.select()` is inferred from the schema. Autocomplete on `users.id`, etc.

8. Auth: Lucia or Better Auth

SvelteKit auth in 2026:

- **Lucia v3** — small, framework-agnostic, you control the storage. Good for fine-grained control.
- **Better Auth** — newer, more batteries-included, more conventions. Less code to write.

I lean **Better Auth** for "just give me sessions + email/password + OAuth" without writing my own. Lucia for "I want full control."

Better Auth setup

```
bun add better-auth
```

`src/lib/server/auth.ts`:

```
import { betterAuth } from 'better-auth';
import { drizzleAdapter } from 'better-auth/adapters/drizzle';
import { db } from '$lib/server/db';
import { AUTH_SECRET } from '$env/static/private';
import { PUBLIC_BASE_URL } from '$env/static/public';

export const auth = betterAuth({
  database: drizzleAdapter(db, { provider: 'pg' }),
  secret: AUTH_SECRET,
  baseURL: PUBLIC_BASE_URL,
  emailAndPassword: {
    enabled: true
  },
  socialProviders: {
    github: {
      clientId: process.env.GITHUB_CLIENT_ID!,
      clientSecret: process.env.GITHUB_CLIENT_SECRET!
    }
  }
});
```

`src/hooks.server.ts`:

```
import { auth } from '$lib/server/auth';
import { svelteKitHandler } from 'better-auth/svelte-kit';

export const handle = svelteKitHandler({ auth });
```

The session is now on `event.locals.user` in every load function / endpoint.

`src/routes/+layout.server.ts`:

```
export const load = async ({ locals }) => {
  return { user: locals.user };
};
```

`src/routes/+page.svelte`:

```
<script lang="ts">
  let { data } = $props();
</script>

{#if data.user}
  <p>Hi {data.user.name}</p>
{:else}
  <a href="/login">Sign in</a>
{/if}
```

That's enough for a working email/password + GitHub OAuth auth flow.

NOTE. BOTH LUCIA AND BETTER AUTH ARE EVOLVING RAPIDLY. Pin minor versions in `package.json` (e.g., `"better-auth": "0.5.x"`), not `^0.5.0`, so a minor bump doesn't surprise you with breaking changes. Re-validate annually.

9. Env vars: build-time vs runtime, public vs private

SvelteKit splits env vars across two axes. This catches people once and then becomes obvious.

Axis	Options
When	<code>static</code> (read at build time, baked in) vs <code>dynamic</code> (read at runtime)
Who	<code>private</code> (server only) vs <code>public</code> (shipped to the browser too)

So four module patterns:

Import path	Read at	Shipped to client
<code>\$env/static/private</code>	Build time	× Server only
<code>\$env/static/public</code>	Build time	□ Inlined in client bundle
<code>\$env/dynamic/private</code>	Runtime	× Server only
<code>\$env/dynamic/public</code>	Runtime	□ Sent on first SSR

Rules

- **Anything `public` is on the client.** Don't put secrets here. The naming convention enforces this: only env vars prefixed with `PUBLIC_` are accessible via the public modules.
`PUBLIC_BASE_URL` ✓, `DATABASE_URL` no.
- **`static` is faster** (compile-time constant) but requires rebuilding the image to change. Useful for things that don't change between deploys (`PUBLIC_BASE_URL` might).
- **`dynamic` is more flexible** (read from `process.env` at runtime). Useful for secrets that rotate without rebuilding.

Typical setup

```
// src/lib/server/db.ts
import { DATABASE_URL } from '$env/static/private';
// or, if DATABASE_URL might change without rebuild:
import { env } from '$env/dynamic/private';
const url = env.DATABASE_URL;

// src/lib/config.ts (shared)
import { PUBLIC_BASE_URL } from '$env/static/public';
```

In Docker

The compose `env_file: .env` injects all variables at runtime. Static vars (`$env/static/*`) are read **at build time** — so they need to be set during `bun run build` in the Dockerfile's build stage. If `PUBLIC_BASE_URL` is static, you'd either:

- Pass it as a `--build-arg` to the docker build, then `ARG` + `ENV` in the Dockerfile, OR
- Move it to `$env/dynamic/public` so it's read at runtime.

For simplicity: **use dynamic** by default. Switch to static only when you've profiled and need the slight perf win.

10. CI: GitHub Actions → GHCR

Same workflow as the Python guide; just `target: runtime` for the SvelteKit build.

`.github/workflows/build.yml`:

```
name: Build and push image

on:
  push:
    branches: [main]
  workflow_dispatch:

jobs:
  build:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write

    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3

      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${GITHUB_ACTOR}
          password: ${GITHUB_TOKEN}

      - id: meta
        uses: docker/metadata-action@v5
        with:
          images: ghcr.io/${GITHUB_REPOSITORY}
          tags: |
            type=raw,value=latest,enable={{is_default_branch}}
            type=sha,prefix=sha-

      - uses: docker/build-push-action@v5
        with:
          context: .
          target: runtime
          push: true
          tags: ${GITHUB_REF_NAME}
          labels: ${GITHUB_REF_NAME}
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

Same `permissions`, same `setup-buildx`, same `metadata-action`. Only difference: `target: runtime` builds the SvelteKit runtime stage. CI doesn't care that the language inside is TypeScript vs Python; Docker handles the polyglot story.

11. CD: deploying to the VPS

`.github/workflows/deploy.yml`:

```

name: Deploy to VPS

on:
  workflow_run:
    workflows: ["Build and push image"]
    types: [completed]
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    if: ${ github.event.workflow_run.conclusion == 'success' }}

    steps:
      - uses: appleboy/ssh-action@v1
        with:
          host: ${ secrets.VPS_HOST }}
          username: deploy
          key: ${ secrets.VPS_SSH_KEY }}
          script: |
            cd ~/my-app
            docker compose pull
            docker compose run --rm migrate
            docker compose up -d app
            docker image prune -f

```

On the VPS, `/home/deploy/my-app/docker-compose.yml`:

```

networks:
  shared:
    external: true

services:
  app:
    image: ghcr.io/joshuawjulian/my-app:latest
    restart: unless-stopped
    env_file: .env
    networks:
      - shared
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "3"

  migrate:
    image: ghcr.io/joshuawjulian/my-app:latest
    profiles: ["migrate"]
    env_file: .env
    networks:
      - shared
    command: bun run db:migrate

```

`/home/deploy/my-app/.env`:

```
DATABASE_URL=postgres://appuser:<real-password>@postgres:5432/appdb
AUTH_SECRET=<64-hex-chars from openssl rand -hex 32>
PUBLIC_BASE_URL=https://app.yourdomain.com
GITHUB_CLIENT_ID=<github oauth>
GITHUB_CLIENT_SECRET=<github oauth>
```

The deploy user runs the same compose dance as the Python apps. The shared Postgres has a user/db for this app (per [vps-from-zero](#) §15).

12. Caddy site block for SvelteKit

In `/home/deploy/infra/Caddyfile`:

```
app.yourdomain.com {
    encode gzip zstd
    reverse_proxy app:3000
}
```

That's it. SvelteKit's adapter-node listens on 3000 by default; Caddy reaches it via the shared network's DNS.

If you want compression handled by Caddy rather than by SvelteKit (recommended): keep `encode gzip zstd` here, and set `precompress: false` in `svelte.config.js` to skip the at-build-time compression step.

If you have static assets served separately for max performance:

```
app.yourdomain.com {
    handle /_app/* {
        # SvelteKit's hashed static assets – cache aggressively
        header Cache-Control "public, max-age=31536000, immutable"
        reverse_proxy app:3000
    }
    handle /static/* {
        # Your /static/ directory
        header Cache-Control "public, max-age=86400"
        reverse_proxy app:3000
    }
    handle {
        reverse_proxy app:3000
    }
}
```

Most personal setups don't need this granularity — the default Caddy reverse_proxy + SvelteKit's own header handling is fine.

13. Troubleshooting

`bun install` works locally but fails in CI

Common: the lockfile is out of date or the host's Bun is a different major version. Pin:

- `oven/bun:1.1-alpine` in Dockerfile.
- Bun version in `package.json` (`"packageManager": "bun@1.1.34"`) so CI uses the same.

"Cannot find module '@sveltejs/adapter-node'"

You probably scaffolded with `adapter-auto` and forgot to install `adapter-node` or remove `adapter-auto`. Confirm:

```
grep adapter svelte.config.js
# import adapter from '@sveltejs/adapter-node'; ← should be this

bun install
```

Vite HMR doesn't update

- WSL2 + repo on `/mnt/c/` — move repo to `/home/<user>/dev/`.
- Container's `node_modules` got clobbered by host's — make sure the anonymous volume override is in compose.
- Vite needs `usePolling: true` in some Docker setups. Add to `vite.config.ts`:

```
export default defineConfig({
  server: {
    watch: {
      usePolling: true
    }
  }
});
```

"ECONNREFUSED postgres:5432"

The app starts before Postgres is ready. Either:

- Confirm `depends_on.postgres.condition: service_healthy` in compose.
- Add reconnect logic in `postgres.js`: `postgres(url, { connect_timeout: 10 })`.

Auth session doesn't persist

- Cookie domain wrong: check `PUBLIC_BASE_URL` matches the actual URL.
- HTTP vs HTTPS mismatch: in dev (`http://localhost:5173`), cookies may need `secure: false` until you switch to HTTPS.
- Behind Caddy: SvelteKit must trust the proxy. Add `csrf: { checkOrigin: true }` in `svelte.config.js` and verify Caddy is passing `X-Forwarded-Host`.

Image is huge (>500MB)

- Dockerfile uses `oven/bun:latest` (debian-based) instead of `oven/bun:1.1-alpine`. Switch to alpine.

- Build stage's `node_modules` is leaking into runtime. Make sure runtime stage explicitly `COPY --from=build /app/node_modules` (production-only after the `bun install --production` step), not from `base` (which has devDeps).
- Forgot to delete `.svelte-kit` and `node_modules` from `.dockerignore`. Check.

CI builds but production crashes on start

Usually env vars. Common slip-ups:

- A `$env/static/private` import is reading at build time, and the build-time `DATABASE_URL` was the local dev value. Either change to dynamic, or pass the prod value as a build arg (less ideal).
- `NODE_ENV` not set to production in the container. Set `ENV NODE_ENV=production` in the runtime stage.

14. Alternatives considered

Runtime

- **Bun** (this guide) — fast, native TS, single binary. Default for new SvelteKit projects.
- **Node.js** + adapter-node — battle-tested, every host supports it. Reconsider when a specific dependency doesn't work on Bun (rare in 2026, but happens).
- **Deno** + adapter-deno — modern, secure-by-default, but smaller SvelteKit community + sometimes friction with npm packages. Skip unless you specifically want Deno's model.

Framework

- **SvelteKit** (this guide) — your default per CLAUDE.md.
- **Next.js** — for React projects. Mature, great DX, gigantic ecosystem. Bigger bundles than SvelteKit.
- **Remix / TanStack Start** — React-flavored, like SvelteKit conceptually. Niche.
- **Astro** — for content-heavy sites. Less interactive; supports islands of Svelte/React/etc.
- **Vanilla Express/Fastify** — when you don't need a frontend. Use Hono on Bun if you do this.

Database client

- **postgres.js** (this guide) + **Drizzle** — fastest, TypeScript-first, Bun-friendly.
- **pg** (node-postgres) — the older choice. Slower than postgres.js. Works.
- **Prisma** — feature-rich ORM. Heavier; query engine is a Rust binary. Reconsider when you want Prisma's introspection and migrations vs Drizzle's schema-first.
- **Kysely** — TypeScript-first query builder, no ORM. Use with postgres.js. Reconsider when Drizzle's ORM patterns aren't fitting.

Auth

- **Better Auth** (this guide) — modern, batteries-included.
- **Lucia v3** — smaller, you control everything. Use when you need fine-grained control.
- **Clerk / Auth0** (hosted) — paid services with great UX. Reconsider when you need MFA-out-of-box, polished UI, and don't mind paying.

- **Roll your own** — possible but high-stakes. The kind of thing where a tiny bug = account takeover. Use a library.

Adapter

- **adapter-node** (this guide) — for self-hosting.
- **adapter-bun** — Bun-specific; faster startup. Compatible with adapter-node mostly, occasional edge cases.
- **adapter-cloudflare / adapter-vercel / adapter-netlify** — for those platforms. Reconsider when you'd rather not run a VPS.
- **adapter-static** — pure static export. Reconsider when your site has no server-side anything (no auth, no DB, no API).

Hosting comparison

Option	When to pick
Your own VPS + Caddy (this guide)	You want control + already have the VPS setup; cheapest per request.
Vercel	Marketing landing pages, free hobby tier, perfect Next.js integration. SvelteKit also supported.
Cloudflare Pages + Workers	Edge deployment, generous free tier, lower latency globally.
Fly.io	Container-based, easy Postgres, regional deploys.
Railway	Simplest "git push to deploy" UX.
Render	Heroku-like ergonomics.

For personal projects you're already hosting on a VPS, **stick with your VPS** unless something forces otherwise — it's already paid for and the workflow's set up.

15. Quick reference

New project setup

```
cd ~/dev
bun create svelte@latest my-app
cd my-app
bun add -d @sveltejs/adapter-node
bun remove @sveltejs/adapter-auto
bun add postgres drizzle-orm better-auth
bun add -d drizzle-kit
git init && git add -A && git commit -m "init"
gh repo create my-app --private --source=. --remote=origin --push
```

Daily dev

```
cd ~/dev/my-app
code . # opens VS Code, prompts "Reopen in Container"
# inside dev container:
bun run dev # http://localhost:5173
bun run check # type check
bun run test # vitest
bun run db:generate # after schema change
bun run db:migrate # apply migrations locally
```

Production checklist (per repo)

- [] `Dockerfile` multi-stage with dev/build/runtime targets.
- [] `.dockerignore` excludes `node_modules`, `.svelte-kit`, `build`, `.env`.
- [] `.env.example` committed.
- [] `svelte.config.js` uses `adapter-node`.
- [] `package.json` pins `"packageManager"` to a specific Bun version.
- [] `bun.lockb` committed.
- [] GHA `build.yml` and `deploy.yml` mirror the Python pipeline.
- [] VPS-side `docker-compose.yml` exists at `/home/deploy/<app>/`.
- [] `.env` on VPS has `DATABASE_URL`, `AUTH_SECRET`, `PUBLIC_BASE_URL`.
- [] Postgres user/db created on the shared infra Postgres.
- [] Caddyfile entry for the app's domain.
- [] DNS pointing at the VPS.

Compose patterns

Need	Command
Start dev locally	<code>docker compose up</code> (after Dev Container start)
Migrate locally	<code>docker compose run --rm migrate</code>
Reset local DB	<code>docker compose down -v && docker compose up -d postgres</code>
Run a one-off Bun script	<code>docker compose run --rm app bun run scripts/seed.ts</code>

Useful Bun commands inside the dev container

```
bun install # install dependencies from lockfile
bun add <pkg> # add a dependency
bun add -d <pkg> # add a dev dependency
bun remove <pkg>
bun run <script> # alias: bun <script> for scripts in package.json
bun test # run tests
bun --bun run dev # force Bun runtime (vs Node-via-shim)
bunx <cli> # like npx
```